Jerome Holland

Dr. Chang
499J
12/2/32025

Description of Dataset and Task

The dataset used for this project is the Pima Indians Diabetes Database (diabetes.arff). This dataset contains diagnostic measurements aimed at predicting the onset of diabetes based on several health factors.

- Source: National Institute of Diabetes and Digestive and Kidney Diseases.
- Total Instances: 768 patient records.
- Task: This is a binary classification task. The objective is to build predictive models to forecast whether a patient will test positive or negative for diabetes.
- Target Variable: class (Two outcomes: tested_positive or tested_negative).
- Features: The dataset includes 8 numeric predictive attributes:

Attribute	Description
preg	Number of times pregnant
plas	Plasma glucose concentration (2-hour post-load)
pres	Diastolic blood pressure (mm Hg)
skin	Triceps skin fold thickness (mm)
insu	2-Hour serum insulin (mu U/ml)
mass	Body mass index (BMI)
pedi	Diabetes pedigree function

age	Age (years)

2. Justification for Choice of Classification Methods

Three classifiers from distinct machine learning families were selected to provide a comparative analysis of model complexity versus performance.

1. Decision Tree (DecisionTreeClassifier): Chosen for its interpretability and ability to model non-linear data without extensive feature engineering. It directly reveals the key diagnostic split points in the data, making the results easy to explain.
2. Naive Bayes (GaussianNB): Chosen as a simple probabilistic baseline model. Its efficiency and speed provide a strong, fast benchmark against which more complex models can be evaluated, despite its strong assumption of feature independence.
3. Neural Network (MLPClassifier): Chosen for its high predictive power and ability to learn complex, non-linear relationships. Its multi-layered architecture is suitable for maximizing classification accuracy on complex biomedical datasets.

3. Data Preprocessing

Data cleaning and transformation were necessary to prepare the dataset for accurate model training, especially for the distance-based and function-based models (MLP).

1. Missing Value Imputation: In this dataset, values of 0 in columns like plas, pres, skin, insu, and mass are often biologically implausible and represent missing data. These and any explicit ? values were replaced with mean imputation for their respective columns.
2. Feature Scaling (Standardization): All eight numerical features were standardized using a StandardScaler. This process transforms the data to have a mean of 0 and a standard deviation of 1. This step is crucial for the Neural Network to ensure that no single feature dominates the learning process due to its magnitude.

4. Experimental Design

- Methodology: 10-Fold Cross-Validation (CV) was used to evaluate model performance, ensuring the results are robust and not dependent on a single, arbitrary train-test split.
- Procedure: The dataset was randomly partitioned into 10 equal subsets. The models were trained and tested 10 times, with each subset serving exactly once as the testing data. The final reported metric is the average performance across these 10 trials.
- Evaluation Metrics: The models were assessed using four key metrics to provide a comprehensive view of performance:
    - Accuracy: The overall percentage of correctly classified instances.

- Precision (Macro-Averaged): Measures the model's ability to avoid false positive classifications.
- Recall (Macro-Averaged): Measures the model's ability to find all positive instances.
- F1-Score (Macro-Averaged): The harmonic mean of Precision and Recall, providing a balanced measure of performance.

---

5. Results

The table below presents the mean scores (± standard deviation) for each classifier across the 10 cross-validation folds.

```
--- Final 10-Fold Cross-Validation Results ---

|                      | Accuracy        | Precision       | Recall          | F1-Score        |
|:---------------------|:----------------|:----------------|:----------------|:----------------|
| Decision Tree        | 0.7538 ± 0.0610 | 0.7326 ± 0.0552 | 0.7249 ± 0.0644 | 0.7222 ± 0.0663 |
| Naive Bayes          | 0.7486 ± 0.0589 | 0.7223 ± 0.0631 | 0.7129 ± 0.0633 | 0.7149 ± 0.0647 |
| Neural Network (MLP) | 0.7435 ± 0.0271 | 0.7164 ± 0.0306 | 0.7126 ± 0.0295 | 0.7127 ± 0.0283 |
PS C:\Users\ronie\Downloads>
```

```python
import pandas as pd
import numpy as np
from scipy.io import arff
from sklearn.preprocessing import StandardScaler

# --- 1. Load and Clean Data ---
# Load the ARFF file
data, meta = arff.loadarff('diabetes.arff')
df = pd.DataFrame(data)

# Convert byte strings to regular strings
df['class'] = df['class'].apply(lambda x: x.decode('utf-8'))

# Imputation: Convert '0' in key columns to NaN and fill with mean
cols_to_impute = ['plas', 'pres', 'skin', 'insu', 'mass']
df[cols_to_impute] = df[cols_to_impute].replace(0, np.nan)
for col in cols_to_impute:
    df[col] = df[col].fillna(df[col].mean())
# --- 2. Feature Scaling and Setup ---
X = df.drop('class', axis=1)
y = df['class']

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
# --- This code prints the final table of results ---
from sklearn.model_selection import KFold, cross_validate
from sklearn.naive_bayes import GaussianNB
from sklearn.neural_network import MLPClassifier

# Setup classifiers
nb_model = GaussianNB()
mlp_model = MLPClassifier(
    random_state=42,
    max_iter=500,
    hidden_layer_sizes=(50, 20),
    tol=1e-4
)

# Setup K-Fold Strategy
cv_strategy = KFold(n_splits=10, shuffle=True, random_state=42)
scoring_metrics = ['accuracy', 'precision_macro', 'recall_macro', 'f1_macro']

# 1. EXECUTION FOR NAIVE BAYES (DEFINES nb_cv_results)
nb_cv_results = cross_validate(
    nb_model,
    X_scaled,
    y,
    cv=cv_strategy,
    scoring=scoring_metrics,
    n_jobs=-1
)

# 2. EXECUTION FOR MLPClassifier (DEFINES mlp_cv_results)
mlp_cv_results = cross_validate(
    mlp_model,
    X_scaled,
    y,
    cv=cv_strategy,
    scoring=scoring_metrics,
    n_jobs=-1
)
from sklearn.tree import DecisionTreeClassifier
```

```python
dt_classifier = DecisionTreeClassifier(random_state=42, max_depth=5)

# 3. EXECUTION FOR DECISION TREE (DEFINES dt_cv_results)
dt_cv_results = cross_validate(
    dt_classifier,
    X_scaled,
    y,
    cv=cv_strategy,
    scoring=scoring_metrics,
    n_jobs=-1
)


# Create a dictsnionary to hold all results
all_results = {}

# Decision Tree results
all_results ['Decision Tree'] = {
    'Accuracy': f"{dt_cv_results['test_accuracy'].mean():.4f} \u00B1 {dt_cv_results['test_accuracy'].std():.4f}",
    'Precision': f"{dt_cv_results['test_precision_macro'].mean():.4f} \u00B1 {dt_cv_results['test_precision_macro'].std():.4f}",
    'Recall': f"{dt_cv_results['test_recall_macro'].mean():.4f} \u00B1 {dt_cv_results['test_recall_macro'].std():.4f}",
    'F1-Score': f"{dt_cv_results['test_f1_macro'].mean():.4f} \u00B1 {dt_cv_results['test_f1_macro'].std():.4f}"
}

# Naive Bayes results
all_results['Naive Bayes'] = {
    'Accuracy': f"{nb_cv_results['test_accuracy'].mean():.4f} \u00B1 {nb_cv_results['test_accuracy'].std():.4f}",
    'Precision': f"{nb_cv_results['test_precision_macro'].mean():.4f} \u00B1 {nb_cv_results['test_precision_macro'].std():.4f}",
    'Recall': f"{nb_cv_results['test_recall_macro'].mean():.4f} \u00B1 {nb_cv_results['test_recall_macro'].std():.4f}",
    'F1-Score': f"{nb_cv_results['test_f1_macro'].mean():.4f} \u00B1 {nb_cv_results['test_f1_macro'].std():.4f}"
}

# MLPClassifier results
all_results['Neural Network (MLP)'] = {
    'Accuracy': f"{mlp_cv_results['test_accuracy'].mean():.4f} \u00B1 {mlp_cv_results['test_accuracy'].std():.4f}",
    'Precision': f"{mlp_cv_results['test_precision_macro'].mean():.4f} \u00B1 {mlp_cv_results['test_precision_macro'].std():.4f}",
    'Recall': f"{mlp_cv_results['test_recall_macro'].mean():.4f} \u00B1 {mlp_cv_results['test_recall_macro'].std():.4f}",
    'F1-Score': f"{mlp_cv_results['test_f1_macro'].mean():.4f} \u00B1 {mlp_cv_results['test_f1_macro'].std():.4f}"
}


# Print the final formatted table using pandas
import pandas as pd
results_df = pd.DataFrame(all_results).T
print("\n--- Final 10-Fold Cross-Validation Results ---\n")
print(results_df.to_markdown())
```

Conclusion

The **Decision Tree Classifier** (Rule-based paradigm) was the most effective predictive model based on mean accuracy, achieving **0.7538 ±0.0610**. This result is particularly significant because the Decision Tree provides the essential benefit of **interpretability**, clearly defining the explicit diagnostic rules used for classification.

The **Naive Bayes** model performed very closely (0.7486 ± 0.0589), providing a strong probabilistic benchmark. The **Neural Network (MLPClassifier)**, while having the lowest mean accuracy (0.7435), showed the highest **stability** with the smallest standard deviation (± 0.0271), indicating its performance is the least sensitive to the specific data folds.